

Task 1

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
int main() {
```

```
    int n, m;
```

```
    cin >> n >> m;
```

```
    string symbols[10];
```

```
    for (int i = 0; i < m; i++) {
```

```
        cin >> symbols[i];
```

```
    }
```

```
    int trans[50][10];
```

```
    for (int i = 0; i < n * m; i++) {
```

```
        int from, to;
```

```
        string sym;
```

```
        cin >> from >> sym >> to;
```

```
        for (int j = 0; j < m; j++) {
```

```
            if (symbols[j] == sym) {
```

```
                trans[from][j] = to;
```

```
            }
```

```
        }
```

```
    }
```

```
int start;  
cin >> start;
```

```
int f;  
cin >> f;  
int finalStates[50] = {0};  
for (int i = 0; i < f; i++) {  
    int x;  
    cin >> x;  
    finalStates[x] = 1;  
}
```

```
int group[50];  
for (int i = 0; i < n; i++) {  
    if (finalStates[i]) {  
        group[i] = 1;  
    } else {  
        group[i] = 0;  
    }  
}
```

```
int changed = 1;  
while (changed) {  
    changed = 0;
```

```
    int newGroup[50];  
    int groupCount = 0;
```

```

for (int i = 0; i < n; i++) {
    int found = -1;
    for (int j = 0; j < i; j++) {
        if (group[i] == group[j]) {
            int same = 1;

            for (int k = 0; k < m; k++) {
                if (group[trans[i][k]] != group[trans[j][k]]) {
                    same = 0;
                    break;
                }
            }

            if (same) {
                found = newGroup[j];
                break;
            }
        }
    }

    if (found == -1) {
        newGroup[i] = groupCount++;
    } else {
        newGroup[i] = found;
    }
}

for (int i = 0; i < n; i++) {
    if (group[i] != newGroup[i]) {

```

```

        changed = 1;
        break;
    }
}

for (int i = 0; i < n; i++) {
    group[i] = newGroup[i];
}
}

cout << "Equivalent Groups:" << endl;

int printed[50] = {0};
for (int i = 0; i < n; i++) {
    if (!printed[i]) {
        cout << "{";
        cout << i;
        printed[i] = 1;

        for (int j = i + 1; j < n; j++) {
            if (group[i] == group[j]) {
                cout << "," << j;
                printed[j] = 1;
            }
        }
        cout << "}" << endl;
    }
}
}

```

```

cout << "Start State = {";
for (int i = 0; i < n; i++) {
    if (group[i] == group[start]) {
        cout << i << ", ";
    }
}
cout << "\b}" << endl;

cout << "Final States:" << endl;

int printedGroup[50] = {0};

for (int i = 0; i < n; i++) {
    if (finalStates[i] && !printedGroup[group[i]]) {
        cout << "{";

        for (int j = 0; j < n; j++) {
            if (group[j] == group[i]) {
                cout << j << ", ";
            }
        }

        cout << "\b}" << endl;
        printedGroup[group[i]] = 1;
    }
}

cout << "Minimized DFA Transition Table:" << endl;

```

```

for (int i = 0; i < n; i++) {
    int isRep = 1;

    for (int j = 0; j < i; j++) {
        if (group[i] == group[j]) {
            isRep = 0;
            break;
        }
    }

    if (!isRep) {
        continue;
    }

    for (int k = 0; k < m; k++) {
        cout << "{";

        for (int j = 0; j < n; j++) {
            if (group[j] == group[i]) {
                cout << j << ",";
            }
        }

        cout << "\b} --" << symbols[k] << "--> {";

        int next = trans[i][k];

        for (int j = 0; j < n; j++) {
            if (group[j] == group[next]) {

```

```

        cout << j << ",";

    }

}

    cout << "\b" << endl;

}

}

return 0;

}

```

```

6 2
a
b
0 a 1
0 b 2
1 a 1
1 b 3
2 a 1
2 b 2
3 a 1
3 b 4
4 a 1
4 b 2
5 a 1
5 b 2
0
1
4
Equivalent Groups:
{0,2,5}
{1}
{3}
{4}
Start State = {0,2,5}
Final States:
{4}
Minimized DFA Transition Table:
{0,2,5} --a--> {1}
{0,2,5} --b--> {0,2,5}
{1} --a--> {1}
{1} --b--> {3}
{3} --a--> {1}
{3} --b--> {4}
{4} --a--> {1}
{4} --b--> {0,2,5}

Process returned 0 (0x0)   execution time : 71.894 s
Press any key to continue.

```

Task 2

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
int main() {
```

```
    int n, m;
```

```
    cin >> n >> m;
```

```
    string symbols[10];
```

```
    for (int i = 0; i < m; i++) {
```

```
        cin >> symbols[i];
```

```
    }
```

```
    int trans[50][10];
```

```
    for (int i = 0; i < n * m; i++) {
```

```
        int from, to;
```

```
        string sym;
```

```
        cin >> from >> sym >> to;
```

```
    for (int j = 0; j < m; j++) {
```

```
        if (symbols[j] == sym) {
```

```
            trans[from][j] = to;
```

```
        }
```



```
    }  
}
```

```
int start;  
cin >> start;
```

```
int f;  
cin >> f;
```

```
int finalStates[50] = {0};
```

```
for (int i = 0; i < f; i++) {  
    int x;  
    cin >> x;  
    finalStates[x] = 1;  
}
```

```
int group[50];
```

```
for (int i = 0; i < n; i++) {  
    if (finalStates[i]) {  
        group[i] = 1;  
    } else {  
        group[i] = 0;  
    }  
}
```

```
int changed = 1;
```

```

while (changed) {
    changed = 0;
    int newGroup[50];
    int groupCount = 0;

    for (int i = 0; i < n; i++) {
        int found = -1;

        for (int j = 0; j < i; j++) {
            if (group[i] == group[j]) {
                int same = 1;

                for (int k = 0; k < m; k++) {
                    if (group[trans[i][k]] != group[trans[j][k]]) {
                        same = 0;
                        break;
                    }
                }

                if (same) {
                    found = newGroup[j];
                    break;
                }
            }
        }

        if (found == -1) {
            newGroup[i] = groupCount++;
        } else {

```

```
        newGroup[i] = found;
    }
}
```

```
for (int i = 0; i < n; i++) {
    if (group[i] != newGroup[i]) {
        changed = 1;
        break;
    }
}
```

```
for (int i = 0; i < n; i++) {
    group[i] = newGroup[i];
}
}
```

```
cout << "Equivalent Groups:" << endl;
```

```
int printed[50] = {0};
```

```
for (int i = 0; i < n; i++) {
    if (!printed[i]) {
        cout << "{";
        cout << i;
        printed[i] = 1;

        for (int j = i + 1; j < n; j++) {
            if (group[i] == group[j]) {
                cout << "," << j;
```

```
        printed[j] = 1;
    }
}
cout << "}" << endl;
}
}
```

```
int startGroup = group[start];
```

```
cout << "Start State = {";
for (int i = 0; i < n; i++) {
    if (group[i] == startGroup) {
        cout << i << ", ";
    }
}
cout << "\b}" << endl;
```

```
cout << "Final States:" << endl;
```

```
int printedGroup[50] = {0};
int isFinalGroup[50] = {0};
```

```
for (int i = 0; i < n; i++) {
    if (finalStates[i]) {
        isFinalGroup[group[i]] = 1;
    }
}
```

```
for (int i = 0; i < n; i++) {
```

```

if (finalStates[i] && !printedGroup[group[i]]) {
    cout << "{";

    for (int j = 0; j < n; j++) {
        if (group[j] == group[i]) {
            cout << j << ", ";
        }
    }

    cout << "\b}" << endl;
    printedGroup[group[i]] = 1;
}
}

cout << "Minimized DFA Transition Table:" << endl;

for (int i = 0; i < n; i++) {
    int isRep = 1;

    for (int j = 0; j < i; j++) {
        if (group[i] == group[j]) {
            isRep = 0;
            break;
        }
    }

    if (!isRep) {
        continue;
    }
}

```

```

for (int k = 0; k < m; k++) {
    cout << "{";
    for (int j = 0; j < n; j++) {
        if (group[j] == group[i]) {
            cout << j << ", ";
        }
    }
    cout << "\b} --" << symbols[k] << "--> {";

    int next = trans[i][k];

    for (int j = 0; j < n; j++) {
        if (group[j] == group[next]) {
            cout << j << ", ";
        }
    }
    cout << "\b}" << endl;
}
}

```

```

int q;
cin >> q;

```

```

while (q--) {
    string str;
    cin >> str;

```

```

    cout << "String: " << str << endl;

```

```
cout << "Path: ";
```

```
int currentGroup = startGroup;
```

```
cout << "{";
```

```
for (int i = 0; i < n; i++) {
```

```
    if (group[i] == currentGroup) {
```

```
        cout << i << ",";
```

```
    }
```

```
}
```

```
cout << "\\b}";
```

```
for (int i = 0; i < str.length(); i++) {
```

```
    char ch = str[i];
```

```
    int symbolIndex = -1;
```

```
    for (int j = 0; j < m; j++) {
```

```
        if (symbols[j][0] == ch) {
```

```
            symbolIndex = j;
```

```
            break;
```

```
        }
```

```
    }
```

```
    int rep = -1;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (group[j] == currentGroup) {
```

```
            rep = j;
```

```
            break;
```

```

    }
}

int nextState = trans[rep][symbolIndex];
currentGroup = group[nextState];

cout << " -> {";
for (int j = 0; j < n; j++) {
    if (group[j] == currentGroup) {
        cout << j << ", ";
    }
}
cout << "\b";
}

cout << endl;

if (isFinalGroup[currentGroup]) {
    cout << "Accepted" << endl;
} else {
    cout << "Rejected" << endl;
}
}

return 0;
}

```



```

2 a 1
2 b 2
3 a 1
3 b 4
4 a 1
4 b 2
5 a 1
5 b 2
0
1
4
Equivalent Groups:
{0,2,5}
{1}
{3}
{4}
Start State = {0,2,5}
Final States:
{4}
Minimized DFA Transition Table:
{0,2,5} --a--> {1}
{0,2,5} --b--> {0,2,5}
{1} --a--> {1}
{1} --b--> {3}
{3} --a--> {1}
{3} --b--> {4}
{4} --a--> {1}
{4} --b--> {0,2,5}
3
ab
String: ab
Path: {0,2,5} -> {1} -> {3}
Rejected
abb
String: abb
Path: {0,2,5} -> {1} -> {3} -> {4}
Accepted
bbb
String: bbb
Path: {0,2,5} -> {0,2,5} -> {0,2,5} -> {0,2,5}
Rejected

Process returned 0 (0x0)   execution time : 20.933 s
Press any key to continue.

```